

LAPORAN TUGAS BESAR 3

IF2211 STRATEGI ALGORITMA

Pemanfaatan *Pattern Matching* untuk Membangun
Sistem ATS (*Applicant Tracking System*) Berbasis CV Digital



Disusun oleh :

Bryan Ho	13523029
Alvin Christopher Santausa	13523033
Carlo Angkisan	13523091

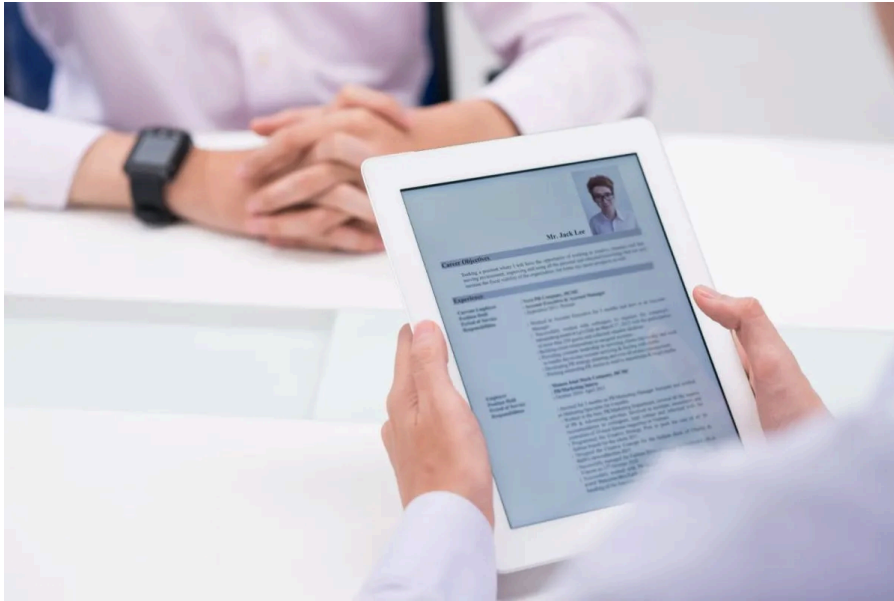
PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	3
BAB II.....	5
2.1 Algoritma Knuth-Morris-Pratt (KMP).....	5
2.2 Algoritma Boyer-Moore (BM).....	5
2.3 Algoritma Aho–Corasick.....	6
2.4 Aplikasi Yang Dibangun.....	7
BAB III.....	8
3.1 Langkah-Langkah Pemecahan Masalah.....	8
3.2 Pemetaan Masalah Menjadi Elemen-Elemen Algoritma.....	9
3.3 Fitur Fungsional dan Arsitektur Aplikasi Web.....	11
3.4 Contoh Ilustrasi Kasus.....	13
BAB IV.....	21
4.1 Spesifikasi Teknis Program.....	21
4.2 Tata Cara Penggunaan Program.....	30
4.3 Hasil Pengujian.....	33
4.4 Analisis Hasil Pengujian.....	40
BAB V.....	42
5.1 Kesimpulan.....	42
5.2 Saran.....	42
5.3 Refleksi.....	42
LAMPIRAN.....	44
Tautan Repository Github.....	44
Tautan Video.....	44
Tabel Kelengkapan Spesifikasi.....	44
DAFTAR PUSTAKA.....	46

BAB I

DESKRIPSI TUGAS



Gambar 1. CV ATS dalam Dunia Kerja
(Sumber: <https://www.antaraneews.com/>)

Di era digital ini, keamanan data dan akses menjadi semakin penting. Perkembangan proses rekrutmen tenaga kerja telah mengalami perubahan signifikan dengan memanfaatkan teknologi untuk meningkatkan efisiensi dan akurasi. Salah satu inovasi yang menjadi solusi utama adalah Applicant Tracking System (ATS), yang dirancang untuk mempermudah perusahaan dalam menyaring dan mencocokkan informasi kandidat dari berkas lamaran, khususnya Curriculum Vitae (CV). ATS memungkinkan perusahaan untuk mengelola ribuan dokumen lamaran secara otomatis dan memastikan kandidat yang relevan dapat ditemukan dengan cepat.

Meskipun demikian, salah satu tantangan besar dalam pengembangan sistem ATS adalah kemampuan untuk memproses dokumen CV dalam format PDF yang tidak selalu terstruktur. Dokumen seperti ini memerlukan metode canggih untuk mengekstrak informasi penting seperti identitas, pengalaman kerja, keahlian, dan riwayat pendidikan secara efisien. Pattern matching menjadi solusi ideal dalam menghadapi tantangan ini.

Pattern matching adalah teknik untuk menemukan dan mencocokkan pola tertentu dalam teks. Dalam konteks ini, algoritma Boyer-Moore dan Knuth-Morris-Pratt (KMP) sering digunakan karena keduanya menawarkan efisiensi tinggi untuk pencarian teks di dokumen besar. Algoritma ini memungkinkan sistem ATS untuk mengidentifikasi informasi penting dari CV pelamar dengan kecepatan dan akurasi yang optimal.

Di dalam Tugas Besar 3 ini, Anda diminta untuk mengimplementasikan sistem yang dapat melakukan deteksi informasi pelamar berbasis dokumen CV digital. Metode yang akan digunakan untuk melakukan deteksi pola dalam CV adalah Boyer-Moore dan Knuth-Morris-Pratt. Selain itu, sistem ini akan dihubungkan dengan identitas kandidat melalui basis data sehingga harapannya terbentuk sebuah sistem yang dapat mengenali profil pelamar secara lengkap hanya dengan menggunakan CV digital.

BAB II

LANDASAN TEORI

2.1 Algoritma *Knuth-Morris-Pratt* (KMP)

Algoritma Knuth-Morris-Pratt (KMP) merupakan salah satu metode yang efisien untuk masalah pencocokan string yang menghindari pemeriksaan ulang terhadap karakter yang sudah dibandingkan sebelumnya. Diperkenalkan oleh Donald Knuth, Vaughan Pratt, dan James Morris pada tahun 1977, KMP bekerja berdasarkan prinsip bahwa informasi dari pencocokan yang telah dilakukan dapat dimanfaatkan untuk mengurangi jumlah perbandingan ketika terjadi mismatch. Untuk itu, KMP membentuk sebuah prefix table, atau lebih dikenal sebagai border function, sebelum proses pencocokan dimulai. Border function ini berisi informasi tentang panjang prefiks pola yang juga merupakan sufiks, untuk setiap posisi dalam pola. Dengan kata lain, fungsi ini memetakan berapa banyak karakter awal dari pola yang bisa "dipertahankan" saat terjadi ketidakcocokan, sehingga algoritma tidak perlu memulai pencocokan dari awal pola.

Selama proses pencocokan, apabila ditemukan perbedaan karakter antara teks dan pola, KMP tidak kembali ke indeks awal dari pola, melainkan langsung melompat ke indeks yang sesuai berdasarkan border function. Teknik ini secara signifikan mengurangi waktu pencarian, karena menghindari perbandingan karakter-karakter yang sudah pasti tidak mungkin cocok. Efisiensi ini membuat setiap karakter dalam teks hanya diperiksa satu kali, yang menyebabkan kompleksitas waktu algoritma menjadi $O(n + m)$, dengan n adalah panjang teks dan m adalah panjang pola. Efisiensi tersebut menjadikan KMP ideal untuk digunakan dalam pencocokan string terhadap teks yang sangat panjang atau dalam aplikasi real-time, seperti pencarian kata dalam stream data.

2.2 Algoritma *Boyer-Moore* (BM)

Algoritma Boyer-Moore merupakan salah satu metode pencocokan string tercepat dalam praktik dan sangat populer untuk digunakan dalam aplikasi-aplikasi pemrosesan teks modern. Dikembangkan oleh Robert S. Boyer dan J Strother Moore pada tahun 1977, algoritma ini dirancang untuk mencari sebuah pola dalam teks dengan melakukan pergeseran cerdas berdasarkan informasi yang diperoleh dari pola itu sendiri. Tidak seperti algoritma KMP yang melakukan pencocokan dari kiri ke kanan, Boyer-Moore justru memulai pencocokan dari kanan ke kiri pattern. Strategi ini memungkinkan algoritma untuk menghindari banyak perbandingan yang tidak perlu dan memberikan efisiensi tinggi dalam situasi praktis, terutama ketika teks memiliki alfabet yang besar.

Salah satu komponen kunci dari algoritma ini adalah last occurrence function, yaitu fungsi yang menyimpan informasi tentang posisi terakhir kemunculan setiap karakter dalam pola. Fungsi ini digunakan untuk menentukan seberapa jauh pola dapat digeser

ketika terjadi ketidakcocokan (mismatch). Jika karakter dalam teks yang dibandingkan tidak terdapat dalam pola, maka pola dapat langsung digeser melewati karakter tersebut. Sebaliknya, jika karakter terdapat dalam pola, algoritma akan menggeser pola sejauh mungkin untuk menyelaraskan kemunculan terakhir karakter tersebut. Dalam kasus terbaik, strategi ini memungkinkan pola melompati sebagian besar teks tanpa perlu diperiksa satu per satu.

Pendekatan ini tidak hanya memungkinkan penghematan besar dalam jumlah perbandingan karakter, tetapi juga menjadikan algoritma Boyer-Moore sangat efisien dalam praktik. Meskipun kompleksitas waktu terburuknya bisa mencapai $O(nm)$, kasus ini jarang terjadi. Dalam sebagian besar aplikasi nyata, Boyer-Moore mampu menyelesaikan pencocokan dengan kompleksitas mendekati $O(n)$, dan performanya jauh mengungguli metode pencocokan lainnya. Oleh karena itu, algoritma ini sangat direkomendasikan untuk digunakan dalam pencarian teks berukuran besar seperti dalam editor teks, pengindeksan dokumen, atau sistem pencarian dokumen legal dan medis.

2.3 Algoritma Aho–Corasick

Algoritma Aho–Corasick adalah algoritma pencocokan string yang dirancang khusus untuk kasus pencarian banyak pola secara bersamaan (multi-pattern matching) dalam satu lintasan teks. Dikembangkan oleh Alfred V. Aho dan Margaret J. Corasick pada tahun 1975, algoritma ini menyatukan kekuatan pencocokan string dan teori automaton deterministik untuk mencapai efisiensi tinggi dalam mendeteksi semua pola yang mungkin muncul dalam teks. Proses utama algoritma ini terdiri dari dua tahap, yaitu tahap pembuatan automaton dan tahap pencocokan teks.

Tahap pertama membangun struktur data berbentuk trie dari semua pola yang ingin dicocokkan. Trie ini kemudian diperkuat dengan fail function, yaitu mekanisme fallback yang memungkinkan pencarian tetap berjalan meskipun karakter yang diharapkan tidak ditemukan. Fail function ini bekerja mirip dengan border function pada algoritma KMP, yaitu mengarahkan automaton ke simpul alternatif terdekat yang masih memungkinkan kecocokan ketika transisi gagal. Dengan bantuan fail function ini, Aho–Corasick mampu menghindari pencocokan ulang karakter-karakter yang sudah diperiksa sebelumnya, sehingga pencocokan menjadi sangat efisien.

Keunggulan utama dari algoritma ini adalah kemampuannya untuk melakukan pencocokan terhadap ribuan pola sekaligus dalam satu kali pembacaan teks, menjadikannya sangat cocok untuk aplikasi berskala besar seperti pemfilteran konten internet, pencarian keyword dalam dokumen hukum atau medis, serta deteksi signature dalam sistem antivirus. Kompleksitas waktu pencocokan Aho–Corasick adalah $O(n + z)$, di mana n adalah panjang teks dan z adalah jumlah total kemunculan pola yang

ditemukan. Meskipun tahap pembentukan automaton memerlukan waktu dan memori tambahan, keuntungan yang diperoleh selama proses pencocokan sangat besar dan jauh mengimbangi biaya awal tersebut. Kemampuan untuk menangani banyak pola secara simultan dengan efisiensi tinggi menjadikan Aho–Corasick sebagai algoritma utama dalam dunia pencarian teks yang kompleks dan besar.

2.4 Aplikasi Yang Dibangun

Aplikasi yang dibangun dalam Tugas Besar 3 ini merupakan sistem Applicant Tracking System (ATS) berbasis digital yang dirancang untuk membantu proses penyaringan kandidat melalui analisis isi dokumen Curriculum Vitae (CV). Aplikasi ini memungkinkan pengguna untuk melakukan pencarian keyword dalam CV menggunakan tiga algoritma pencocokan pola, yaitu Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), dan Aho-Corasick. Pengguna dapat memasukkan sejumlah keyword yang dipisahkan dengan koma, memilih algoritma yang akan digunakan, serta menentukan jumlah hasil teratas yang ingin ditampilkan melalui slider interaktif. Hasil pencarian ditampilkan dalam bentuk daftar kandidat yang diurutkan berdasarkan skor relevansi, lengkap dengan informasi seperti jumlah keyword yang cocok, total kemunculan kata, dan nilai kecocokan.

Aplikasi ini dibangun menggunakan Python sebagai bahasa pemrograman utama dengan antarmuka pengguna berbasis CustomTkinter, sebuah pustaka GUI modern berbasis tkinter yang mendukung desain responsif dan bergaya kontemporer. Selain itu, digunakan juga MySQL sebagai sistem manajemen basis data untuk menyimpan data CV kandidat, serta pustaka-pustaka tambahan seperti PyPDF2 untuk ekstraksi teks dari file PDF, dotenv untuk pengelolaan konfigurasi lingkungan, dan Faker untuk proses seeding data uji.

Struktur arsitektur aplikasi memisahkan antara logika bisnis di backend dan tampilan pengguna di frontend. Modul CVService bertugas menangani pemuatan data, pencarian, dan perhitungan skor, sedangkan modul PatternMatcher mengimplementasikan berbagai algoritma pencocokan string. Secara keseluruhan, aplikasi ini dirancang untuk mempermudah proses evaluasi kandidat secara cepat, akurat, dan terstruktur, dengan dukungan teknologi yang fleksibel dan dapat dikembangkan lebih lanjut.

BAB III

ANALISIS PEMECAHAN MASALAH

3.1 Langkah-Langkah Pemecahan Masalah

Pendekatan pemecahan masalah dalam proyek ini dirancang secara sistematis untuk menangani tantangan dalam pencarian kandidat dari kumpulan Curriculum Vitae (CV) berformat PDF. Proses ini mencakup ekstraksi data, pencarian kata kunci dengan berbagai metode, hingga penyajian hasil yang relevan kepada pengguna.

1. Ekstraksi dan Pengolahan Data CV

Proses dimulai dengan pengolahan setiap file CV berformat PDF. Parsing dilakukan sebanyak dua kali untuk setiap dokumen.

- **Parsing Pertama (Ekstraksi Ringkasan)**
Regular Expression (Regex) digunakan untuk mengidentifikasi dan mengekstrak bagian-bagian kunci dari CV, seperti "Skills", "Professional Experience", dan "Education". Informasi ini kemudian disimpan secara terstruktur untuk digunakan dalam fitur ringkasan (summary).
- **Parsing Kedua (Normalisasi Teks)**
Seluruh konten teks dari PDF diekstrak dan diubah menjadi sebuah continuous string atau untaian teks tunggal yang panjang. String ini kemudian dinormalisasi (misalnya, diubah ke huruf kecil) untuk menghilangkan ambiguitas dan memfasilitasi proses pencocokan pola yang efisien.

2. Penerapan Algoritma Pattern Matching

Setelah pra-pemrosesan, sistem melakukan pencarian kata kunci (keyword) yang dimasukkan pengguna menggunakan pendekatan dua fase untuk memastikan hasil yang komprehensif.

- **Fase 1: Pencarian Eksak (Exact Matching)**
Pengguna dapat memilih salah satu dari tiga algoritma pencocokan string yang telah diimplementasikan: Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), atau Aho-Corasick. Algoritma yang dipilih akan mencari kecocokan persis (exact match) antara kata kunci yang diberikan dan continuous string dari setiap CV. Hasil dari fase ini, termasuk jumlah kecocokan dan kata kunci apa yang ditemukan, akan dicatat.
- **Fase 2: Pencarian Kabur (Fuzzy Matching) sebagai Fallback**

Jika pencarian eksak tidak menemukan kecocokan sama sekali atau jumlah hasil yang ditemukan kurang dari yang diharapkan pengguna, sistem secara otomatis beralih ke mode pencarian kabur. Metode yang digunakan adalah Levenshtein Distance. Tujuannya adalah untuk menemukan kata-kata dalam CV yang mirip atau mendekati kata kunci, meskipun tidak identik secara sempurna. Ini sangat berguna untuk menangani kesalahan ketik (typo) atau variasi istilah dalam CV.

3. Penilaian dan Peringkat CV

Setiap CV diberi skor relevansi berdasarkan hasil pencarian. Skor ini dihitung berdasarkan jumlah total kecocokan, yang merupakan kombinasi dari kecocokan eksak dan kabur. CV dengan skor tertinggi akan ditempatkan di urutan teratas, sehingga pengguna dapat dengan cepat mengidentifikasi kandidat yang paling relevan.

4. Penyajian Hasil

Hasil pencarian yang telah diperingkat disajikan kepada pengguna melalui antarmuka grafis (GUI). Setiap kandidat ditampilkan dalam bentuk kartu (card) yang berisi informasi ringkas seperti nama pelamar, jumlah total kata kunci yang cocok, dan daftar kata kunci yang ditemukan. Pengguna juga diberikan opsi untuk melihat ringkasan (summary) CV yang telah diekstrak atau membuka file PDF asli secara langsung dari aplikasi.

3.2 Pemetaan Masalah Menjadi Elemen-Elemen Algoritma

Masalah utama dalam pattern matching adalah menemukan satu atau lebih kata kunci (pola) di dalam dokumen teks yang panjang (konten CV). Pemetaan masalah ini ke dalam kerangka kerja algoritma terbagi menjadi 2, yaitu konten CV yang telah diekstraksi dan dinormalisasi menjadi teks (text) dan setiap kata kunci yang dicari oleh pengguna berfungsi sebagai pola (pattern).

1. Algoritma Knuth-Morris-Pratt (KMP)

KMP memecahkan masalah ini dengan memindai teks dari kiri ke kanan. Keunggulan utamanya adalah penggunaan tabel bantu yang disebut failure function (atau border function). Tabel ini dibangun dari pola sebelum pencarian dimulai dan berisi informasi tentang awalan terpanjang dari pola yang juga merupakan sufiks. Ketika terjadi ketidakcocokan karakter antara teks dan pola, failure function memungkinkan algoritma untuk menggeser pola secara cerdas tanpa perlu memundurkan pointer pada teks, sehingga menghindari perbandingan karakter yang berulang dan meningkatkan efisiensi.

2. Algoritma Boyer-Moore (BM)

BM menerapkan pendekatan yang berbeda dengan memulai perbandingan dari karakter paling kanan pada pola. Strategi ini sering kali lebih cepat dalam praktiknya karena didukung oleh dua aturan heuristik yang kuat, yaitu

- **Bad Character Rule**
Ketika terjadi ketidakcocokan, aturan ini menggunakan posisi kemunculan terakhir dari karakter yang tidak cocok di dalam pola untuk menentukan sejauh mana pergeseran dapat dilakukan.
- **Good Suffix Rule**
Jika sebagian sufiks dari pola sudah cocok dengan teks, aturan ini menggunakan informasi tersebut untuk menentukan pergeseran yang aman dan optimal. Dengan menggabungkan kedua aturan ini, BM sering kali dapat melakukan pergeseran yang sangat besar, melompati sebagian besar teks dalam satu langkah.

3. Algoritma Aho-Corasick

Ketika sistem perlu mencari banyak kata kunci secara bersamaan, KMP dan BM yang melakukannya satu per satu menjadi tidak efisien. Di sinilah Aho-Corasick unggul. Algoritma ini menggabungkan semua pola (kata kunci) ke dalam satu struktur data terpadu berbentuk Trie (pohon prefiks). Setelah Trie dibangun, failure links ditambahkan ke setiap node menggunakan pendekatan Breadth-First Search (BFS). Mirip dengan failure function pada KMP, failure links ini mengubah Trie menjadi sebuah mesin keadaan terbatas (finite automaton). Hal ini memungkinkan Aho-Corasick untuk menemukan semua kemunculan dari semua kata kunci hanya dalam satu kali proses pembacaan teks CV, menjadikannya solusi yang sangat efisien untuk pencarian multi-pola.

4. Algoritma Levenshtein Distance (untuk Fuzzy Matching)

Untuk mengatasi kata kunci yang tidak ditemukan secara eksak, Levenshtein Distance digunakan. Algoritma ini tidak mencari kecocokan persis, melainkan mengukur "jarak" atau perbedaan antara dua string. Jarak ini didefinisikan sebagai jumlah minimum operasi single-character edit (penyisipan, penghapusan, atau substitusi) yang diperlukan untuk mengubah satu string menjadi string lainnya.

- **Implementasi**
Untuk setiap kata kunci, algoritma ini menghitung Levenshtein Distance terhadap setiap kata dalam teks CV. Proses ini dilakukan menggunakan pendekatan dynamic programming, biasanya dengan membangun matriks

(atau dioptimalkan dengan dua array) untuk menyimpan biaya transformasi.

- **Thresholding**

Setelah jarak dihitung, nilai tersebut dinormalisasi (biasanya dibagi dengan panjang kata yang lebih panjang) untuk mendapatkan rasio kemiripan. Rasio ini kemudian dibandingkan dengan ambang batas (threshold) yang telah ditentukan, yaitu 0.7. Nilai 0.7 merupakan titik tengah yang ideal antara presisi (menemukan hanya kata yang benar-benar mirip) dan perolehan (menemukan semua kemungkinan kata yang relevan). Dengan ambang batas 0.7, sistem ini secara efektif dapat mentolerir kesalahan pengetikan manusia yang paling umum terjadi (yaitu kesalahan/typo pada satu huruf). Jika rasio kemiripan melebihi threshold, kata tersebut dianggap sebagai fuzzy match dan dicatat sebagai hasil.

3.3 Fitur Fungsional dan Arsitektur Aplikasi Web

Fitur-fitur utama yang disediakan antara lain:

1. Pencarian CV dengan Keyword

Pengguna dapat memasukkan serangkaian keyword yang dipisahkan koma (contoh: "Tubes 3 Stima, Mtring Satching") untuk dicari di seluruh basis data CV yang tersedia.

2. Pemilihan Algoritma Pencocokan Pola

Tersedia tiga algoritma pencocokan pola yang dapat dipilih, yaitu KMP (Knuth-Morris-Pratt), BM (Boyer-Moore), dan Aho-Corasick.

3. Pengaturan Jumlah Hasil

Pengguna dapat dengan mudah mengatur jumlah hasil teratas yang ingin ditampilkan menggunakan slider interaktif, dengan opsi untuk menampilkan semua hasil yang cocok.

4. Tampilan Hasil yang Terurut dan Informatif

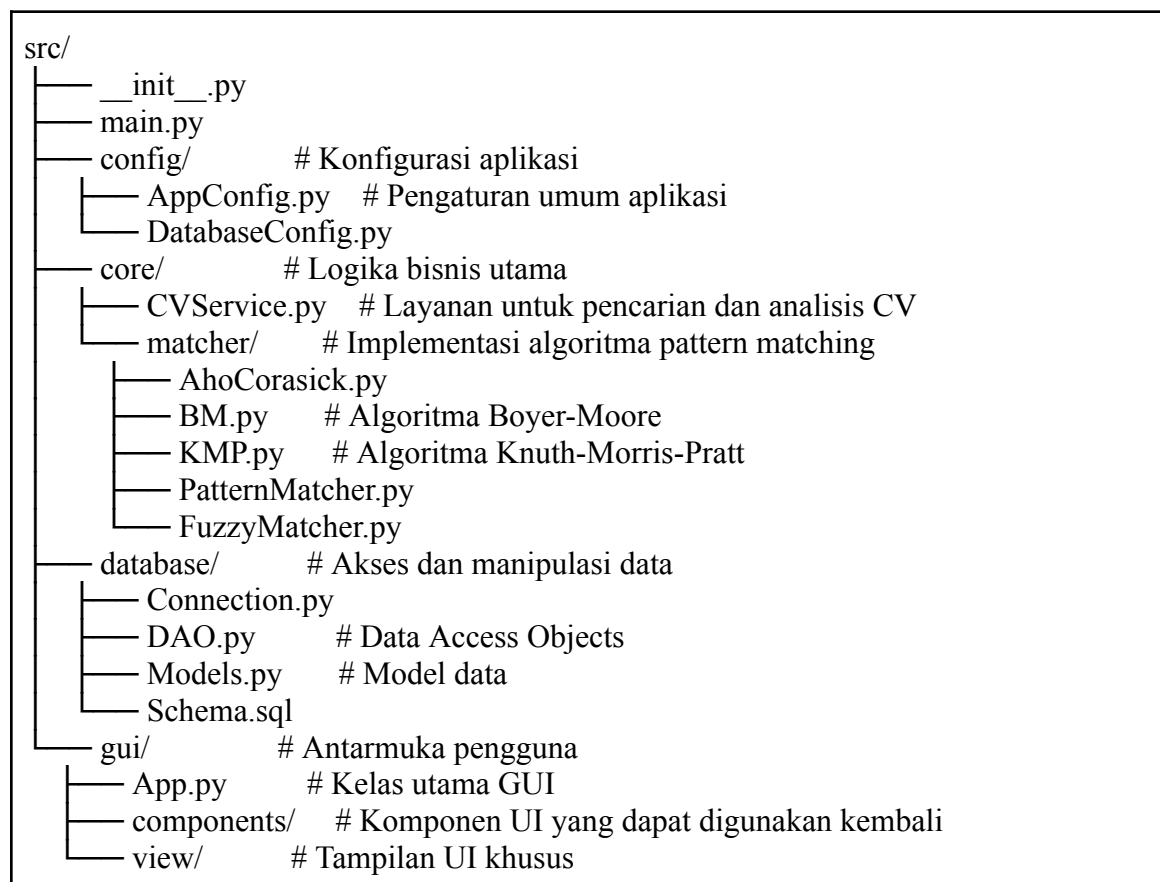
Hasil pencarian disajikan dalam bentuk daftar kartu yang diurutkan berdasarkan skor relevansi tertinggi. Setiap kartu menampilkan nama kandidat, jumlah total kata kunci yang cocok, daftar kata kunci spesifik yang ditemukan, dan skor kecocokan.

5. Detail dan CV *summary*

Setiap kartu hasil memiliki dua tombol interaktif, yaitu:

- Lihat Ringkasan (View Summary)
Membuka tampilan yang berisi ringkasan terstruktur dari CV, mencakup informasi personal, keahlian, pengalaman kerja, dan riwayat pendidikan yang diekstrak menggunakan Regex.
- Lihat CV (View CV)
Membuka file PDF asli dari CV kandidat langsung di dalam aplikasi, lengkap dengan fitur scroll dan paginasi.

Arsitektur utama aplikasi adalah sebagai berikut :



Arsitektur aplikasi ini memisahkan antara antarmuka pengguna (GUI) dengan logika bisnis di backend. GUI dibangun menggunakan library customtkinter untuk tampilan yang modern dan responsif. Di sisi backend, terdapat CVService yang bertanggung jawab atas proses bisnis utama, seperti memuat data CV dari database, menjalankan pencarian, dan menghitung skor. Untuk pencarian itu sendiri, aplikasi menggunakan modul PatternMatcher yang mengimplementasikan berbagai algoritma pencocokan string.

3.4 Contoh Ilustrasi Kasus

Berikut adalah contoh hasil ekstraksi file CV ke dalam bentuk *lowercase* dan *continuous* untuk memudahkan pencarian.

senior digital marketing strategist with over 8 years of experience in digital campaigns, brand development and content optimization. thrives in fast-paced environments and adapts quickly to changing trends. highly analytical with strong communication and project management skills. areas of expertise brand positioning digital advertising campaign development seo/sem strategy content creation market research performance metrics client relations cross-functional leadership creative direction data analysis adobe creative suite google analytics crm software email marketing work experience marketing manager january 2021 to current company name city , state lead end-to-end execution of integrated marketing campaigns across social, email, and search platforms increasing lead generation by 35% y-o-y developed and managed \$400k annual marketing budget ensuring 12% cost savings via strategic vendor negotiations coordinated launch of 3 product lines with go-to-market strategy, achieving 120% of target revenue within first quarter implemented new crm workflows that increased customer retention rate by 18% directed a team of 5 marketers and 3 designers to meet campaign goals content marketing specialist march 2018 to december 2020 company name city , state created seo-optimized blog and web content resulting in 250% increase in organic traffic within one year conducted a/b testing on email campaigns and landing pages leading to 21% higher conversion rates collaborated with sales team to align messaging and drive inbound strategy maintained editorial calendar and produced 10+ monthly articles, whitepapers, and infographics analyzed kpis using google analytics and semrush to iterate content strategy marketing coordinator june 2015 to february 2018 company name city , state assisted in planning and executing corporate events, webinars, and conferences supported digital team with paid media setup, reporting, and vendor communication helped increase social media engagement by 70% through strategic content curation and scheduling education bachelor of arts in marketing university of california los angeles 2011 - 2015 certifications google ads certified hubspot content marketing certified email marketing certification - digital marketer

Contoh ilustrasi kasus adalah pencarian dengan kata kunci (pattern) “digital”. Proses pattern matching untuk algoritma KMP dan BM akan diilustrasikan di bawah ini.

1. Algoritma KMP

Tabel hasil border/failure function dari pola (pattern) yang ingin dicari.

Pattern: digital

j	0	1	2	3	4	5	6
p[j]	d	i	g	i	t	a	l

k	-	0	1	2	3	4	5
b(k)	-	0	0	0	0	0	0

Proses pencarian akan diilustrasikan dengan penemuan pertama yaitu pada “senior digital marketing ...”.

Iterasi pertama:

string	s	e	n	i	o	r		d	i
pattern	d								
i	0								
j	0								

Terjadi *mismatch*, tapi karena indeks j berada pada angka 0, maka merujuk pada tabel *border/failure function*, pola perulangan pada indeks 0 tidak ada, sehingga geser 1 posisi.

Iterasi kedua:

string	e	n	i	o	r		d	i	g
pattern	d								
i	0								
j	0								

Terjadi *mismatch*, tapi karena indeks j berada pada angka 0, maka merujuk pada tabel *border/failure function*, pola perulangan pada indeks 0 tidak ada, sehingga geser 1 posisi.

Iterasi ketiga:

string	n	i	o	r		d	i	g	i
pattern	d								

i	0								
j	0								

Terjadi *mismatch*, tapi karena indeks j berada pada angka 0, maka merujuk pada tabel *border/failure function*, pola perulangan pada indeks 0 tidak ada, sehingga geser 1 posisi.

Iterasi keempat:

string	i	o	r		d	i	g	i	t
pattern	d								
i	0								
j	0								

Terjadi *mismatch*, tapi karena indeks j berada pada angka 0, maka merujuk pada tabel *border/failure function*, pola perulangan pada indeks 0 tidak ada, sehingga geser 1 posisi.

Iterasi kelima:

string	o	r		d	i	g	i	t	a
pattern	d								
i	0								
j	0								

Terjadi *mismatch*, tapi karena indeks j berada pada angka 0, maka merujuk pada tabel *border/failure function*, pola perulangan pada indeks 0 tidak ada, sehingga geser 1 posisi.

Iterasi keenam:

string	r		d	i	g	i	t	a	l
pattern	d								
i	0								
j	0								

Terjadi *mismatch*, tapi karena indeks j berada pada angka 0, maka merujuk pada tabel *border/failure function*, pola perulangan pada indeks 0 tidak ada, sehingga geser 1 posisi.

Iterasi ketujuh:

string		d	i	g	i	t	a	l	
pattern	d								
i	0								
j	0								

Terjadi *mismatch*, tapi karena indeks j berada pada angka 0, maka merujuk pada tabel *border/failure function*, pola perulangan pada indeks 0 tidak ada, sehingga geser 1 posisi.

Iterasi kedelapan:

string	d	i	g	i	t	a	l		m
pattern	d								
i	0								
j	0								

Terjadi *match* sehingga i dan j akan bertambah 1 posisi.

Iterasi kesembilan:

string	d	i	g	i	t	a	l		m
pattern	d	i							
i		1							
j		1							

Terjadi *match* sehingga i dan j akan bertambah 1 posisi.

Iterasi kesepuluh:

string	d	i	g	i	t	a	l		m
--------	---	---	---	---	---	---	---	--	---

pattern	d	i	g						
i			2						
j			2						

Terjadi *match* sehingga i dan j akan bertambah 1 posisi.

Iterasi kesebelas:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i					
i				3					
j				3					

Terjadi *match* sehingga i dan j akan bertambah 1 posisi.

Iterasi kedua-belas:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t				
i					4				
j					4				

Terjadi *match* sehingga i dan j akan bertambah 1 posisi.

Iterasi ketiga-belas:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a			
i						5			
j						5			

Terjadi *match* sehingga i dan j akan bertambah 1 posisi.

Iterasi keempat-belas:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		
i							6		
j							6		

Pada iterasi keempat-belas, Terjadi *match* dan tidak ada sisa huruf pada pattern, sehingga pattern berhasil ditemukan pada text. Jumlah pengecekan yang terjadi adalah sebanyak 14 kali.

2. Algoritma BM

Tabel hasil *last occurrence* dari pola (pattern) yang ingin dicari.

Pattern: digital

x	d	i	g	t	a	l	lainnya
$l(x)$	0	3	2	4	5	6	-1

Proses pencarian akan diilustrasikan dengan penemuan pertama yaitu pada “senior digital marketing ...”.

Iterasi pertama:

string	s	e	n	i	o	r		d	i
pattern	d	i	g	i	t	a	l		
i							6		
j							6		

Terjadi *mismatch*, pada indeks ke-6 dari i dan j, tapi karena pada tabel *last occurrence* tidak terdapat “white space” yang ditandai dengan -1, maka pattern akan bergeser sejauh $i + 1$.

Iterasi kedua:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		
i							6		

j							6		
---	--	--	--	--	--	--	---	--	--

Terjadi *match* sehingga i dan j akan berkurang 1 posisi.

Iterasi ketiga:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		
i						5			
j						5			

Terjadi *match* sehingga i dan j akan berkurang 1 posisi.

Iterasi keempat:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		
i					4				
j					4				

Terjadi *match* sehingga i dan j akan berkurang 1 posisi.

Iterasi kelima:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		
i				3					
j				3					

Terjadi *match* sehingga i dan j akan berkurang 1 posisi.

Iterasi keenam:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		

i			2						
j			2						

Terjadi *match* sehingga i dan j akan berkurang 1 posisi.

Iterasi ketujuh:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		
i		1							
j		1							

Terjadi *match* sehingga i dan j akan bertambah 1 posisi.

Iterasi kedelapan:

string	d	i	g	i	t	a	l		m
pattern	d	i	g	i	t	a	l		
i	0								
j	0								

Pada iterasi kedelapan, Terjadi *match* dan tidak ada sisa huruf pada pattern, sehingga pattern berhasil ditemukan pada text. Jumlah pengecekan yang terjadi adalah sebanyak 8 kali.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Spesifikasi Teknis Program

1. Struktur Data

Struktur Data	Penjelasan
<code>TrieNode = { 'children': Dict[str, 'TrieNode'], 'failure': 'TrieNode', 'outputs': List[str] }</code>	Struktur data ini berupa node dalam Trie. children untuk navigasi karakter, outputs untuk menyimpan pattern yang berakhir di node tersebut, dan failure adalah link untuk transisi saat karakter tidak cocok.
<code>bad_char_table: Dict[str, int]</code>	Sebuah dictionary yang menyimpan indeks kemunculan terakhir dari setiap karakter dalam pattern. Ini digunakan untuk menentukan jarak pergeseran saat terjadi ketidakcocokan (mismatch).
<code>good_suffix_table: List[int]</code>	Sebuah array yang menyimpan jarak pergeseran berdasarkan bagian dari pattern yang sudah cocok (good suffix). Aturan ini diterapkan saat terjadi ketidakcocokan.
<code>failure_function: List[int]</code>	Sebuah array (juga dikenal sebagai tabel LPS) yang menyimpan panjang prefiks terpanjang dari pattern yang sekaligus merupakan sufiks. Ini membantu algoritma menggeser pattern secara efisien setelah mismatch.
<code>SearchResult = Dict[str, List[int]]</code>	Struktur data hasil pencarian yang mengelompokkan temuan. Key adalah pattern yang cocok, dan value adalah list yang berisi semua posisi (indeks) awal dari temuan tersebut.
<code>MatchPositions = List[int]</code>	Struktur data hasil pencarian yang berupa list yang berisi semua indeks awal di mana pattern ditemukan di dalam teks.
<code>FuzzyMatch = List[Tuple[str, float, int]]</code>	Hasil pencarian fuzzy yang berisi daftar tuple. Setiap tuple menyimpan kata

	yang cocok, rasio kemiripan (similarity ratio), dan posisi kata tersebut di dalam teks.
--	---

2. Fungsi dan Prosedur

a. Knuth-Morris-Pratt (KMP)

build_failure_function
Membangun fungsi kegagalan (border function) untuk algoritma KMP. Border function adalah awalan (prefix) terpanjang yang juga merupakan akhiran (suffix).
Kode <pre>def build_failure_function(pattern: str) -> List[int]: """Membangun fungsi kegagalan (border array) untuk algoritma KMP""" if not pattern: return [] m = len(pattern) failure = [0] * m j = 0 # panjang prefiks-sufiks terpanjang sebelumnya # Menghitung failure[i] untuk i = 1 sampai m-1 for i in range(1, m): # pattern[i] tidak cocok dengan pattern[j] while j > 0 and pattern[i] != pattern[j]: j = failure[j - 1] # pattern[i] cocok dengan pattern[j] if pattern[i] == pattern[j]: j += 1 failure[i] = j return failure</pre>

search
Melakukan pencarian string menggunakan algoritma Knuth-Morris-Pratt (KMP). Algoritma KMP meningkatkan efisiensi pencarian dengan memanfaatkan informasi tentang kecocokan sebelumnya untuk menghindari perbandingan yang tidak perlu. KMP menggunakan fungsi kegagalan (failure function) yang telah dibangun sebelumnya untuk menentukan berapa banyak karakter yang dapat dilewati saat terjadi ketidakcocokan.
Kode <pre>def search(text: str, pattern: str) -> List[int]:</pre>

```

"""Mencari pola dalam teks menggunakan algoritma KMP"""
if not text or not pattern:
    return []

# Konversi ke huruf kecil untuk pencarian tidak peka huruf besar/kecil
text = text.lower()
pattern = pattern.lower()

n = len(text)
m = len(pattern)

if m > n:
    return []

matches = []
failure = KMPMatcher.build_failure_function(pattern)

i = 0 # indeks untuk teks
j = 0 # indeks untuk pola

while i < n:
    if pattern[j] == text[i]:
        i += 1
        j += 1

    if j == m:
        # Menemukan kecocokan
        matches.append(i - j)
        j = failure[j - 1] # Dapatkan posisi berikutnya untuk diperiksa
    elif i < n and pattern[j] != text[i]:
        # Ketidakcocokan setelah j kecocokan
        if j != 0:
            j = failure[j - 1]
        else:
            i += 1

return matches

```

count_occurrences

Menghitung jumlah kemunculan pola (pattern) dalam teks.

Kode

```

def count_occurrences(text: str, pattern: str) -> int:
    """Count the number of occurrences of pattern in text"""
    return len(KMPMatcher.search(text, pattern))

```

b. Algoritma Boyer-Moore (BM)

build_bad_character_table

Membangun tabel karakter buruk menggunakan kemunculan terakhir (*last occurrence*) untuk algoritma Boyer-Moore. Tabel ini memungkinkan untuk melewati karakter ketika ketidakcocokan terjadi dengan memanfaatkan dua heuristik, yaitu bad character atau *last occurrence* dan good suffix untuk menentukan berapa banyak langkah yang dapat dilewati.

```
Kode
def build_bad_character_table(pattern: str) -> Dict[str, int]:
    """Membangun tabel karakter buruk untuk algoritma Boyer-Moore"""
    table = {}
    m = len(pattern)

    # Simpan kemunculan paling kanan dari setiap karakter
    for i in range(m):
        table[pattern[i]] = i

    return table
```

build_good_suffix_table

Membangun tabel sufiks baik untuk algoritma Boyer-Moore. Tabel ini membantu menentukan seberapa jauh untuk menggeser ketika sufiks dari pola cocok.

```
Kode
def build_good_suffix_table(pattern: str) -> List[int]:
    """Membangun tabel sufiks baik untuk algoritma Boyer-Moore"""
    m = len(pattern)
    good_suffix = [m] * m

    # Hitung array border
    border = [0] * m
    i = m
    j = m + 1
    border[i - 1] = j

    while i > 1:
        while j <= m and pattern[i - 1] != pattern[j - 1]:
            if good_suffix[j - 1] == m:
                good_suffix[j - 1] = j - i
            j = border[j - 1]
        i -= 1
        j -= 1
        border[i - 1] = j

    # Isi entri yang tersisa
    j = border[0]
    for i in range(m):
        if good_suffix[i] == m:
```

```

    good_suffix[i] = j
    if i == j:
        j = border[j]

return good_suffix

```

search

Melakukan pencarian string menggunakan algoritma Boyer-Moore. Algoritma ini bekerja dengan mencocokkan pola dari kanan ke kiri dan dapat melewati beberapa karakter pada saat ketidakcocokan ditemukan.

```

Kode
def search(text: str, pattern: str) -> List[int]:
    """Mencari pola dalam teks menggunakan algoritma Boyer-Moore

    Args:
        text: Teks yang akan dicari
        pattern: Pola yang dicari

    Returns:
        Daftar indeks di mana pola ditemukan dalam teks
    """
    if not text or not pattern:
        return []

    # Konversi ke huruf kecil untuk pencarian tidak peka huruf besar/kecil
    text = text.lower()
    pattern = pattern.lower()

    n = len(text)
    m = len(pattern)

    if m > n:
        return []

    # Bangun tabel bad character dan good suffix
    bad_char = BoyerMooreMatcher.build_bad_character_table(pattern)

    matches = []
    i = 0 # indeks untuk teks

    while i <= n - m:
        j = m - 1 # mulai mencocokkan dari akhir pattern

        # Cocokkan karakter dari kanan ke kiri
        while j >= 0 and pattern[j] == text[i + j]:
            j -= 1

        if j < 0:
            # Pola ditemukan
            matches.append(i)

```

```

# Geser untuk pencarian berikutnya
i += 1
else:
    # Karakter tidak cocok, gunakan aturan bad character
    char = text[i + j]
    # Menggunakan bad character rule untuk pergeseran
    skip = j - bad_char.get(char, -1)
    i += max(1, skip)

return matches

```

count_occurrences

Menghitung jumlah kemunculan pola (pattern) dalam teks.

```

Kode
def count_occurrences(text: str, pattern: str) -> int:
    """Count the number of occurrences of pattern in text"""
    return len(BoyerMooreMatcher.search(text, pattern))

```

c. Algoritma Aho–Corasick

build_trie

Membangun struktur trie dari daftar pola. Ini menciptakan struktur dasar untuk automaton Aho-Corasick yang memungkinkan pencarian multi-pola secara efisien.

```

Kode
def build_trie(self, patterns: Iterable[str]):
    """Membangun struktur trie dari daftar pola"""
    for pattern in patterns:
        current_node = self.root

        for char in pattern:
            if char not in current_node.children:
                current_node.children[char] = self.TreeNode()
            current_node = current_node.children[char]

        current_node.outputs.append(pattern)

```

build_failure_links

Membangun tautan kegagalan untuk automaton Aho-Corasick menggunakan BFS. Tautan ini memungkinkan traversal efisien ketika ketidakcocokan karakter terjadi.

```

Kode
def build_failure_links(self):

```

```

"""Membangun tautan kegagalan untuk automaton Aho-Corasick menggunakan BFS"""
queue = deque()

# Tetapkan tautan kegagalan untuk simpul kedalaman 1 ke root
for char, child in self.root.children.items():
    child.failure = self.root
    queue.append(child)

# BFS untuk menetapkan tautan kegagalan untuk sisa simpul
while queue:
    current = queue.popleft()

    for char, child in current.children.items():
        queue.append(child)

    failure = current.failure

    while failure is not None and char not in failure.children:
        failure = failure.failure

    if failure is None:
        child.failure = self.root
    else:
        child.failure = failure.children[char]
    # Tambahkan output dari simpul kegagalan ke simpul ini
    child.outputs.extend(child.failure.outputs)

```

search

Mencari semua kemunculan pola dalam teks menggunakan trie yang telah dibangun.

```

Kode
def search(self, text: str) -> Dict[str, List[int]]:
    """Mencari semua kemunculan pola dalam teks"""
    result: Dict[str, List[int]] = {}
    current_node = self.root

    for i in range(len(text)):
        char = text[i]

        # Ikuti tautan kegagalan sampai kita menemukan transisi yang valid
        while current_node != self.root and char not in current_node.children:
            current_node = current_node.failure

        # Buat transisi jika mungkin
        if char in current_node.children:
            current_node = current_node.children[char]

        # Proses kecocokan pola pada posisi ini
        for pattern in current_node.outputs:
            position = i - len(pattern) + 1

```

```

if pattern not in result:
    result[pattern] = []
    result[pattern].append(position)

return result

```

d. Algoritma Levenshtein

distance

Menghitung jarak Levenshtein antara dua string menggunakan dynamic programming. Fungsi ini menentukan jumlah minimum operasi edit (insert, delete, substitute) yang diperlukan untuk mengubah string pertama menjadi string kedua. Hasil berupa integer yang semakin kecil menandakan string semakin mirip.

```

Kode
def distance(s1: str, s2: str) -> int:
    """Hitung jarak Levenshtein antara dua string"""
    if not s1:
        return len(s2)
    if not s2:
        return len(s1)

    # Konversi ke lowercase untuk perbandingan case-insensitive
    s1, s2 = s1.lower(), s2.lower()

    # Buat matrix DP
    rows, cols = len(s1) + 1, len(s2) + 1
    dp = [[0] * cols for _ in range(rows)]

    # Inisialisasi baris dan kolom pertama
    for i in range(rows):
        dp[i][0] = i
    for j in range(cols):
        dp[0][j] = j

    # Isi matrix dengan algoritma DP
    for i in range(1, rows):
        for j in range(1, cols):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] # Tidak ada perubahan
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],    # hapus
                    dp[i][j-1],    # tambah
                    dp[i-1][j-1]    # ganti
                )

    return dp[rows-1][cols-1]

```


similarity_ratio
Mengkonversi jarak Levenshtein menjadi rasio kesamaan antara 0.0-1.0. Menggunakan formula $1.0 - (\text{distance} / \text{max_length})$ dimana 1.0 berarti string identik dan 0.0 berarti sangat berbeda.
Kode <pre>def similarity_ratio(s1: str, s2: str) -> float: """Hitung rasio kesamaan (0.0-1.0) antara dua string""" if not s1 and not s2: return 1.0 if not s1 or not s2: return 0.0 max_len = max(len(s1), len(s2)) distance = LevenshteinMatcher.distance(s1, s2) return 1.0 - (distance / max_len)</pre>

find_similar_words
Mencari kata-kata dalam list yang mirip dengan target word berdasarkan threshold similarity ratio. Mengembalikan list tuple (kata, similarity_ratio) yang diurutkan dari yang paling mirip.
Kode <pre>def find_similar_words(target: str, word_list: List[str], threshold: float = 0.7) -> List[Tuple[str, float]]: """Cari kata-kata mirip dalam list berdasarkan threshold""" matches = [] for word in word_list: ratio = LevenshteinMatcher.similarity_ratio(target, word) if ratio >= threshold: matches.append((word, ratio)) # Urutkan berdasarkan kesamaan tertinggi return sorted(matches, key=lambda x: x[1], reverse=True)</pre>

fuzzy_search_in_text
Melakukan pencarian fuzzy pattern dalam teks panjang dengan mengekstrak kata-kata menggunakan regex. Mengembalikan list tuple (kata_cocok, similarity_ratio, posisi) untuk semua kata yang memenuhi threshold.
Kode <pre>def fuzzy_search_in_text(pattern: str, text: str, threshold: float = 0.7) -> List[Tuple[str, float, int]]: """Cari pattern dalam teks dengan fuzzy matching""" if not pattern or not text: return [] import re</pre>

```

words = re.findall(r'\b\w+\b', text.lower()) # Ekstrak kata-kata
matches = []

for word in words:
    ratio = LevenshteinMatcher.similarity_ratio(pattern.lower(), word)
    if ratio >= threshold:
        position = text.lower().find(word) # Cari posisi dalam teks
        matches.append((word, ratio, position))

# Urutkan berdasarkan kesamaan tertinggi
return sorted(matches, key=lambda x: x[1], reverse=True)

```

4.2 Tata Cara Penggunaan Program

Berikut akan dijelaskan langkah-langkah untuk menjalankan program.

1. Clone Repositori

Clone repositori ke komputer lokal dengan perintah berikut:

```

git clone https://github.com/carllix/Tubes3_MtringSatching.git
cd Tubes3_MtringSatching

```

2. Buat File .env

Di direktori utama proyek, buat file bernama .env dan isi konfigurasi database seperti berikut:

```

DB_HOST=alamat_host_database_anda
DB_PORT=port_database_anda
DB_USER=username_database_anda
DB_PASSWORD=password_database_anda
DB_NAME=nama_database_anda
BASE_DATA_PATH=data/

```

3. Buat Virtual Environment Python

Untuk mengisolasi dependensi proyek, buat dan aktifkan virtual environment:

- Jika menggunakan Windows:

```

python -m venv .env
.venv\Scripts\activate

```

- Jika menggunakan macOS/Linux:

```

python -m venv .env
source .venv/bin/activate

```

4. Instalasi Dependensi

Instal seluruh library Python yang dibutuhkan menggunakan perintah:

```

pip install -r requirements.txt

```

5. Inisialisasi Skema Database

Pastikan MySQL sudah berjalan. Jalankan perintah berikut untuk mengatur struktur tabel di database:

```
mysql -u username_mysql_anda -p < src/database/schema.sql
```

6. Seeding Database

Untuk keperluan pengujian atau pengembangan, pengguna dapat mengisi data sampel ke dalam database dengan:

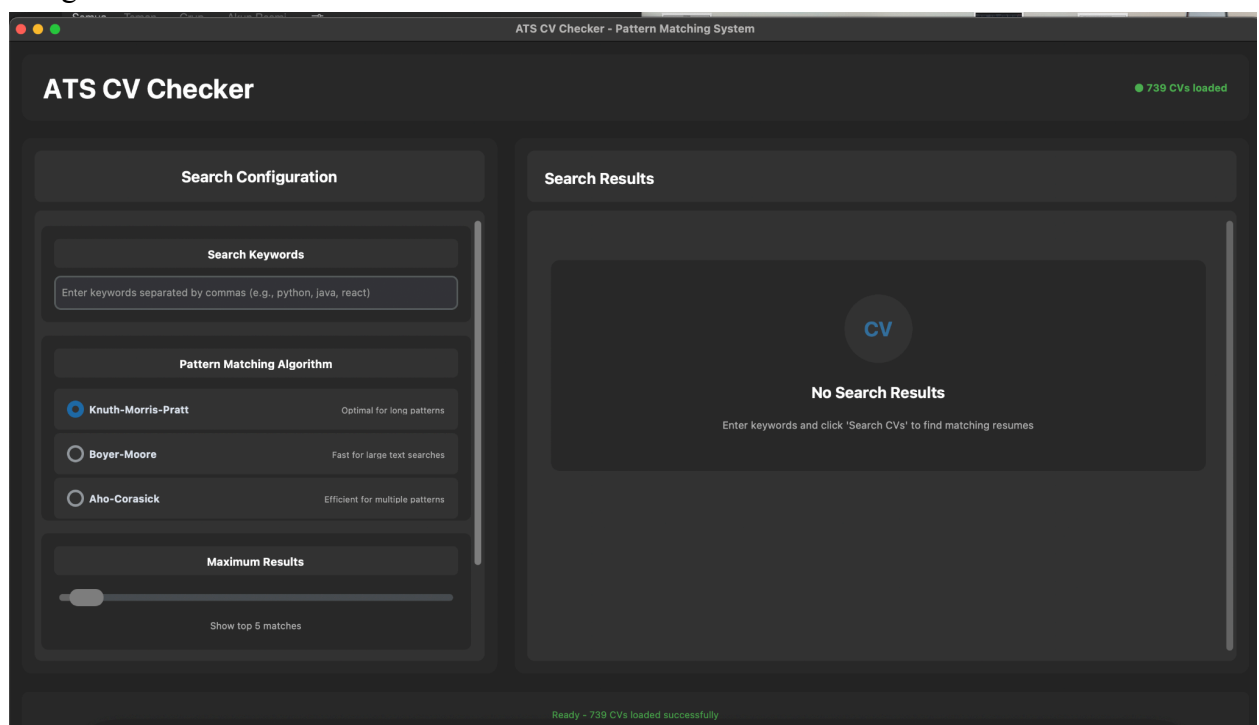
```
python -m src.database.Seeder
```

7. Jalankan Program

Setelah semua langkah selesai, jalankan program utama dengan perintah:

```
python -m src.main
```

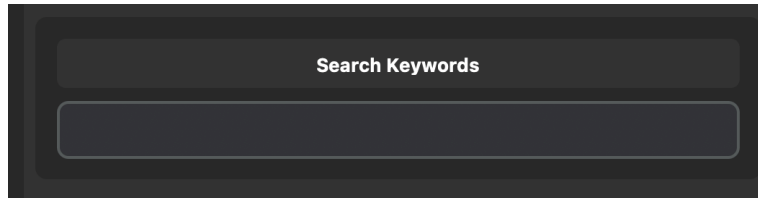
Setelah berhasil menjalankan program, pengguna akan melihat tampilan aplikasi sebagai berikut:



Berikut adalah tata cara penggunaan aplikasi ATS CV Checker:

1. Masukkan Kata Kunci (Keywords).

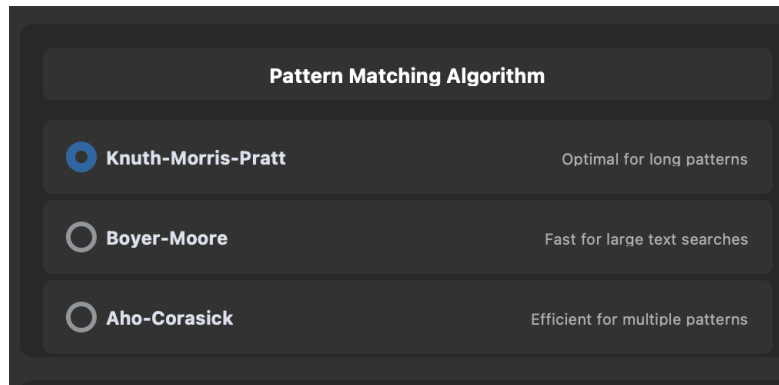
Pada bagian "Keywords", tuliskan satu atau lebih keyword yang ingin dicari, dipisahkan dengan koma. Aplikasi akan mencari semua keyword tersebut pada setiap CV yang tersedia.



2. Pilih Algoritma Pencocokan Pola

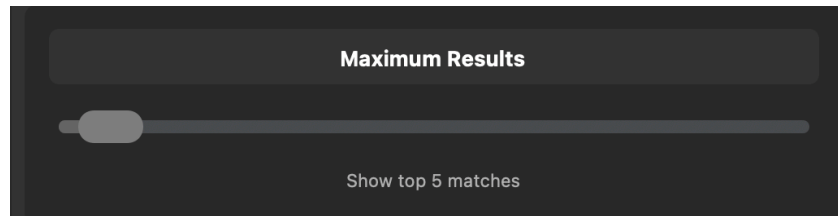
Terdapat tiga algoritma pencocokan pola yang tersedia dan dapat dipilih sesuai kebutuhan, antara lain :

- KMP (Knuth-Morris-Pratt)
- BM (Boyer-Moore)
- Aho-Corasick



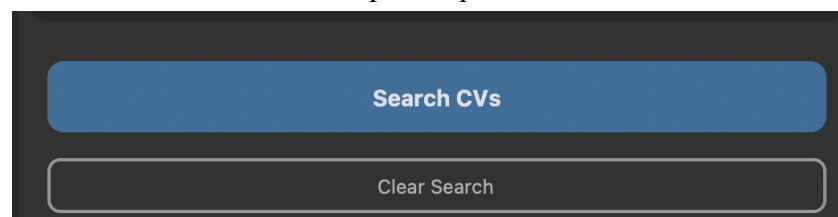
3. Atur Jumlah Hasil yang Ingin Ditampilkan

Gunakan slider interaktif di bawah tulisan "Maximum Results" untuk menentukan jumlah hasil pencarian yang ingin ditampilkan.



4. Klik "🔍 Search CVs"

Setelah mengisi keyword, memilih algoritma, dan mengatur jumlah hasil, klik tombol "Search CVs" untuk memulai proses pencarian.

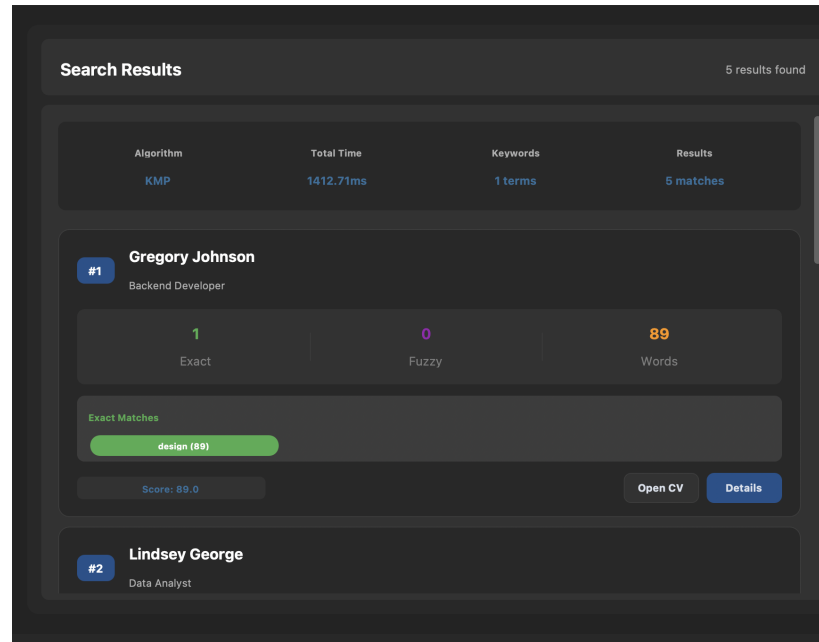


5. Lihat Hasil Pencarian

Hasil akan muncul di panel "Search Results", berisi daftar CV Matches yang diurutkan berdasarkan skor relevansi tertinggi.

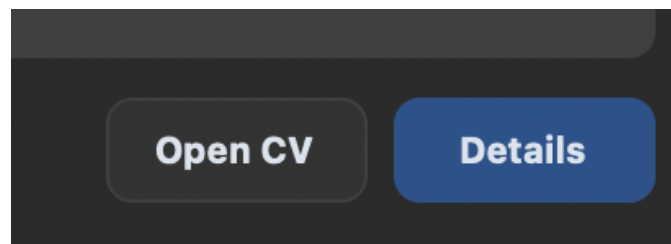
Informasi yang ditampilkan untuk setiap CV meliputi:

- Nama kandidat
- Jumlah keyword yang cocok
- Total kemunculan keyword
- Skor kecocokan (relevansi)



6. Lihat Detail dan Ringkasan CV

Untuk menampilkan ringkasan isi CV secara langsung di aplikasi, klik tombol “Details”. Apabila pengguna ingin membuka file PDF asli dari CV kandidat, klik tombol “Open CV”.



4.3 Hasil Pengujian

Pengujian ini dilakukan untuk menentukan algoritma pencocokan pola yang paling efektif dalam mencari keyword pada dokumen CV. Uji coba dilakukan menggunakan 5 skenario pencarian dengan variasi pada keyword, jenis algoritma, dan panjang keyword yang berbeda-beda. Algoritma yang diuji meliputi Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), dan Aho–Corasick.

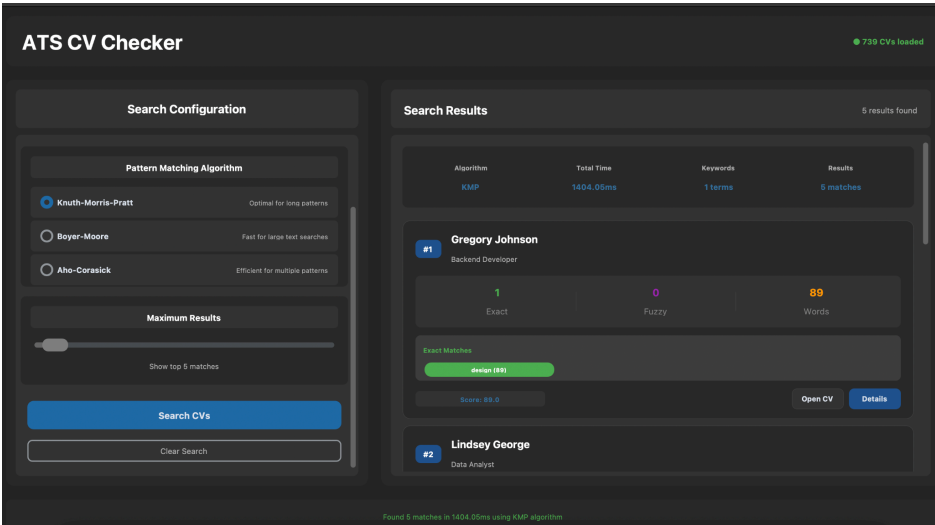
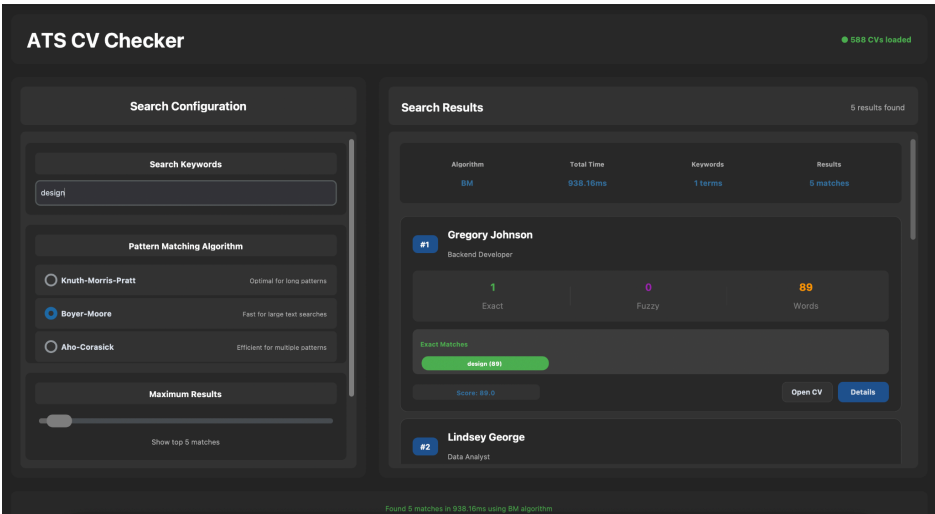
Tabel 1 Hasil Pengujian 1

Kondisi:

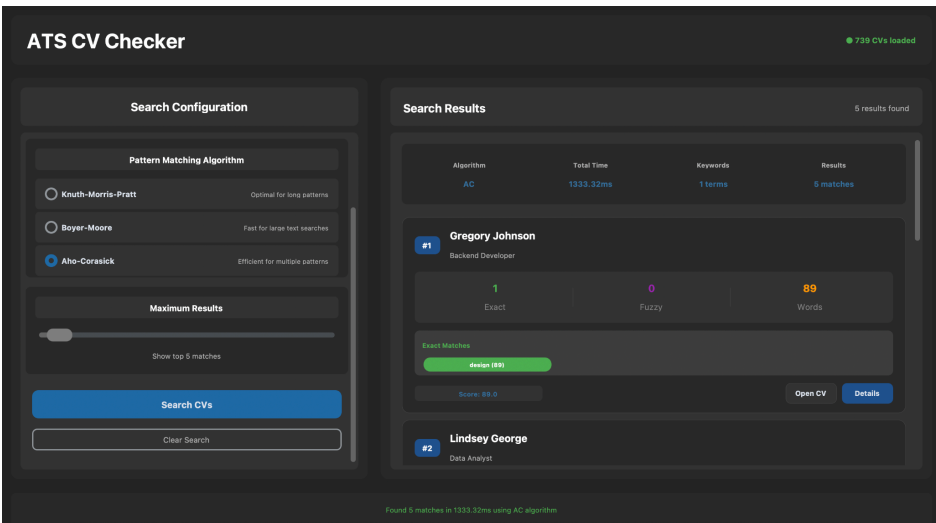
Pencarian dengan Exact String Matching dengan jumlah output yang tertampil dengan jumlah input top matches

Input:
Keyword: design
Top Matches: 5

Output:

Algoritma	Hasil
Knuth-Morris-Pratt	
Boyer-Moore	

Aho-Corasick



Tabel 2 Hasil Pengujian 2

Kondisi: Pencarian dengan keyword typo untuk menguji Levenshtein Distance	
Input: Keyword: desgn Top Matches: 5	
Output:	
Algoritma	Hasil
Knuth-Morris-Pratt	The screenshot shows the 'ATS CV Checker' application with 'Pattern Matching Algorithm' set to 'Knuth-Morris-Pratt' (selected). The 'Search Results' panel shows 5 results found. The top result is for 'Justin Hall', a Data Analyst, with a score of 0.4. The second result is for 'Sean Diaz', a Software Engineer, with a score of 0.4. The interface also displays statistics: 2032.88ms total time, 1 keyword, and 5 matches.

Boyer-Moore	
Aho-Corasick	

Tabel 3 Hasil Pengujian 3

Kondisi: Pencarian dengan keyword dua kata dipisahkan spasi	
Input: Keyword: billing statements Top Matches: 2	
Output:	
Algoritma	Hasil

Knuth-Morris-Pratt

ATS CV Checker

588 CVs loaded

Search Configuration

Pattern Matching Algorithm

☒ Knuth-Morris-Pratt

Optimal for long patterns

☐ Boyer-Moore

Fast for large text searches

☐ Aho-Corasick

Efficient for multiple patterns

Maximum Results

Show top 2 matches

Search CVs

Clear Search

Search Results

2 results found

Algorithm

Total Time

Keywords

Results

KMP

5433.64ms

1 terms

2 matches

#1 Ashley Lozano

QA Tester

1

Exact

0

Fuzzy

1

Words

Exact Matches

Missing statements

Score: 1.0

Open CV

Details

#2 Karen Cherry

System Administrator

Found 2 matches in 5433.64ms using KMP algorithm

Boyer-Moore

ATS CV Checker

588 CVs loaded

Search Configuration

Pattern Matching Algorithm

☐ Knuth-Morris-Pratt

Optimal for long patterns

☒ Boyer-Moore

Fast for large text searches

☐ Aho-Corasick

Efficient for multiple patterns

Maximum Results

Show top 2 matches

Search CVs

Clear Search

Search Results

2 results found

Algorithm

Total Time

Keywords

Results

BM

5252.45ms

1 terms

2 matches

#1 Ashley Lozano

QA Tester

1

Exact

0

Fuzzy

1

Words

Exact Matches

Missing statements

Score: 1.0

Open CV

Details

#2 Karen Cherry

System Administrator

Found 2 matches in 5252.45ms using BM algorithm

Aho-Corasick

ATS CV Checker

588 CVs loaded

Search Configuration

Pattern Matching Algorithm

☐ Knuth-Morris-Pratt

Optimal for long patterns

☐ Boyer-Moore

Fast for large text searches

☒ Aho-Corasick

Efficient for multiple patterns

Maximum Results

Show top 2 matches

Search CVs

Clear Search

Search Results

2 results found

Algorithm

Total Time

Keywords

Results

AC

5409.04ms

1 terms

2 matches

#1 Ashley Lozano

QA Tester

1

Exact

0

Fuzzy

1

Words

Exact Matches

Missing statements

Score: 1.0

Open CV

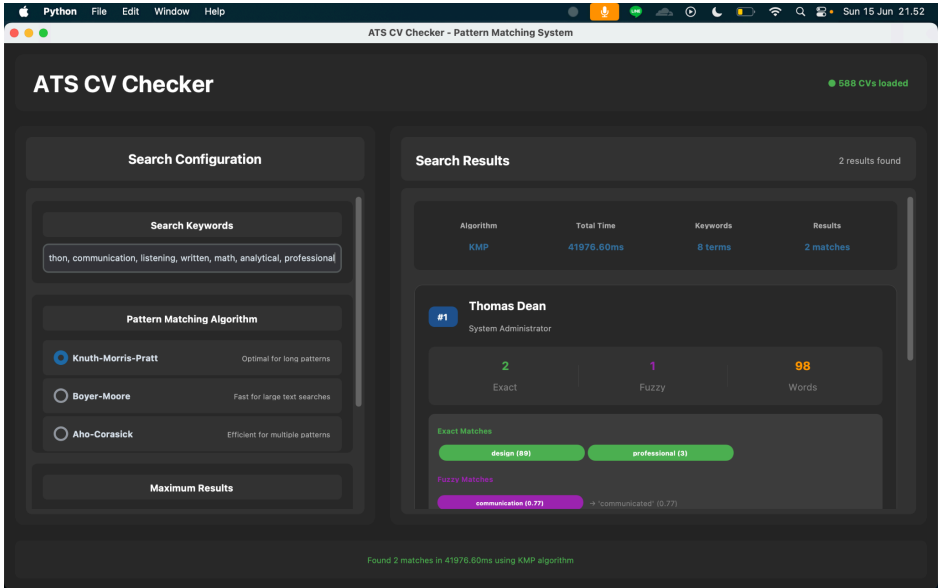
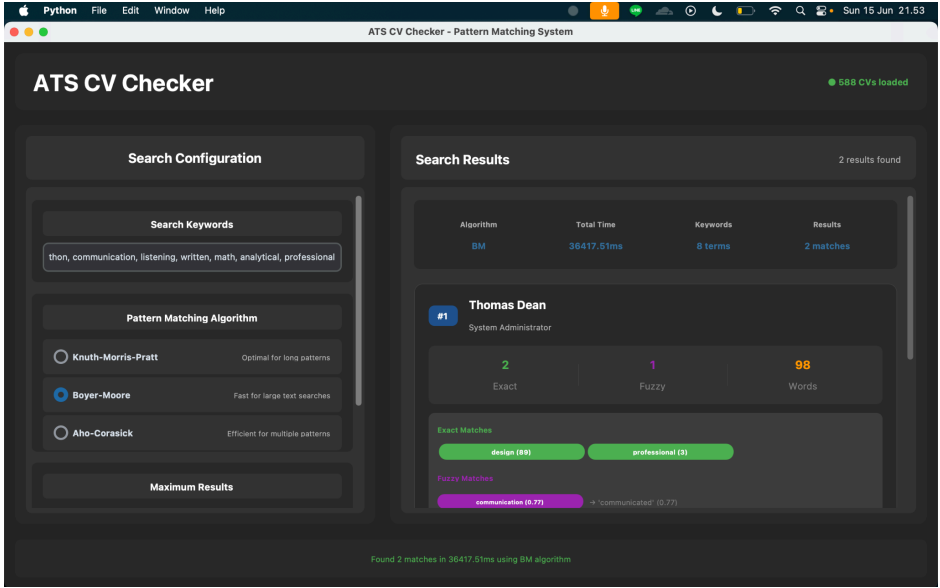
Details

#2 Karen Cherry

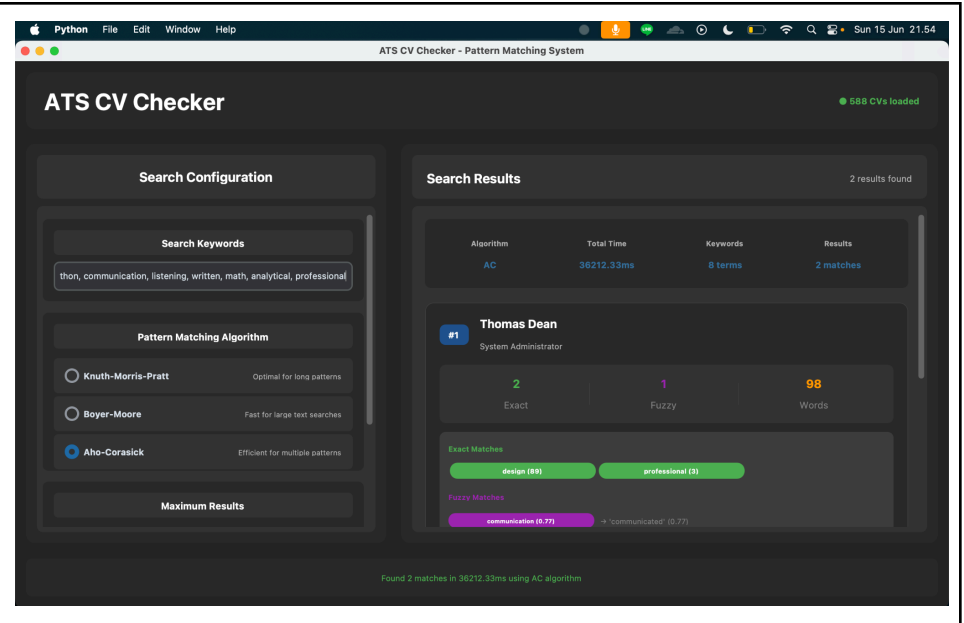
System Administrator

Found 2 matches in 5409.04ms using AC algorithm

Tabel 4 Hasil Pengujian 4

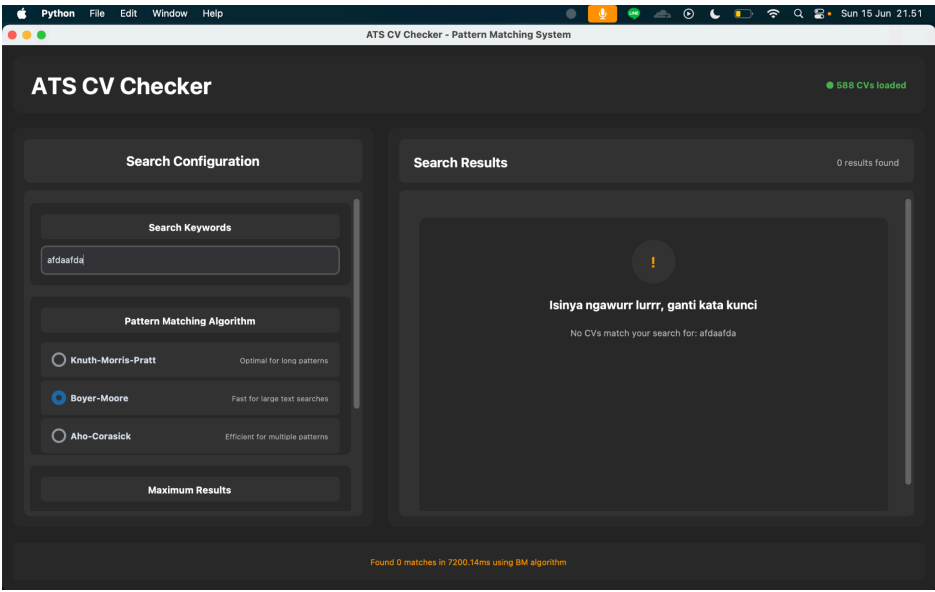
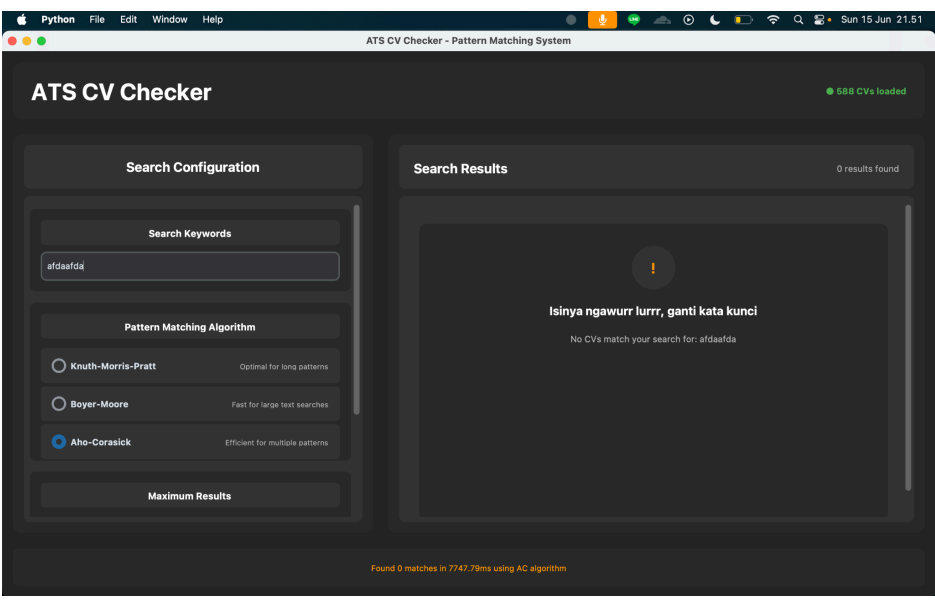
Kondisi: Pencarian dengan delapan keyword	
Input: Keyword: design, python, communication, listening, written, math, analytical, professional Top Matches: 2	
Output:	
Algoritma	Hasil
Knuth-Morris-Pratt	 The screenshot shows the ATS CV Checker interface. On the left, the 'Search Configuration' panel includes a 'Search Keywords' field with the text 'thon, communication, listening, written, math, analytical, professional'. Below it, the 'Pattern Matching Algorithm' section has three options: 'Knuth-Morris-Pratt' (selected), 'Boyer-Moore', and 'Aho-Corasic'. The 'Maximum Results' field is set to 2. On the right, the 'Search Results' panel shows '2 results found'. A table lists the search results: Algorithm (KMP), Total Time (41976.60ms), Keywords (8 terms), and Results (2 matches). Below the table, the first result is for 'Thomas Dean', a System Administrator. It shows 2 exact matches, 1 fuzzy match, and 98 words. The exact matches are 'design' and 'professional'. The fuzzy match is 'communication'. The words are listed as 'communication', 'communication', and 'communication'.
Boyer-Moore	 The screenshot shows the ATS CV Checker interface. On the left, the 'Search Configuration' panel includes a 'Search Keywords' field with the text 'thon, communication, listening, written, math, analytical, professional'. Below it, the 'Pattern Matching Algorithm' section has three options: 'Knuth-Morris-Pratt', 'Boyer-Moore' (selected), and 'Aho-Corasic'. The 'Maximum Results' field is set to 2. On the right, the 'Search Results' panel shows '2 results found'. A table lists the search results: Algorithm (BM), Total Time (36417.51ms), Keywords (8 terms), and Results (2 matches). Below the table, the first result is for 'Thomas Dean', a System Administrator. It shows 2 exact matches, 1 fuzzy match, and 98 words. The exact matches are 'design' and 'professional'. The fuzzy match is 'communication'. The words are listed as 'communication', 'communication', and 'communication'.

Aho-Corasick



Tabel 5 Hasil Pengujian 5

Kondisi: Pencarian dengan tidak ada keyword	
Input: Keyword: afdaafda Top Matches: 5	
Output:	
Algoritma	Hasil
Knuth-Morris-Pratt	The screenshot shows the 'ATS CV Checker - Pattern Matching System' interface. On the left, the 'Search Configuration' panel has 'Search Keywords' set to 'afdaafda', 'Pattern Matching Algorithm' set to 'Knuth-Morris-Pratt' (selected), and 'Maximum Results' set to 5. On the right, the 'Search Results' panel shows '0 results found'. A message in the center says 'Isinya ngawurr lurr, ganti kata kunci' and 'No CVs match your search for: afdaafda'. A status bar at the bottom indicates 'Found 0 matches in 10688.60ms using KMP algorithm'.

Boyer-Moore	 The screenshot shows the 'ATS CV Checker' application interface. On the left, under 'Search Configuration', the 'Search Keywords' field contains 'afdaafda'. Under 'Pattern Matching Algorithm', the 'Boyer-Moore' option is selected. The 'Search Results' panel on the right displays a message: 'Isinya ngawurr lurrr, ganti kata kunci' and 'No CV's match your search for: afdaafda'. At the bottom, it states 'Found 0 matches in 7200.14ms using BM algorithm'.
Aho-Corasick	 The screenshot shows the 'ATS CV Checker' application interface. On the left, under 'Search Configuration', the 'Search Keywords' field contains 'afdaafda'. Under 'Pattern Matching Algorithm', the 'Aho-Corasick' option is selected. The 'Search Results' panel on the right displays the same message: 'Isinya ngawurr lurrr, ganti kata kunci' and 'No CV's match your search for: afdaafda'. At the bottom, it states 'Found 0 matches in 7747.79ms using AC algorithm'.

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan terhadap lima skenario pencarian dengan variasi keyword dan metode pencocokan, dapat disimpulkan bahwa setiap algoritma memiliki keunggulan dalam kondisi tertentu. Algoritma Boyer-Moore (BM) menunjukkan performa paling efisien dari segi waktu eksekusi saat digunakan untuk pencarian dengan satu keyword pada dokumen yang relatif panjang. Hal ini disebabkan oleh strategi pencocokan dari kanan ke kiri dan penggunaan last occurrence function yang memungkinkan algoritma untuk melakukan lompatan besar ketika terjadi ketidakcocokan karakter. Namun, efektivitas Boyer-Moore menjadi terbatas ketika

dihadapkan pada pencarian multi-keyword, karena algoritma ini harus mengulang proses pencocokan untuk setiap keyword secara terpisah.

Sebaliknya, algoritma Aho–Corasick unggul secara signifikan pada skenario pencarian dengan banyak keyword sekaligus, sebagaimana terlihat pada pengujian keempat. Keunggulan ini berasal dari kemampuannya membangun struktur trie yang menggabungkan semua pola pencarian, disertai dengan fail function yang memungkinkan transisi efisien saat terjadi mismatch. Proses pencocokan dapat dilakukan hanya dengan satu lintasan terhadap teks, sehingga menjadikan Aho–Corasick sangat optimal untuk skenario pencarian berskala besar dan kompleks, seperti penyaringan awal CV dengan sejumlah besar kriteria.

Sementara itu, algoritma Knuth-Morris-Pratt (KMP) memiliki keunggulan pada kondisi pencarian dengan satu keyword yang relatif pendek dan dokumen yang tidak terlalu besar. KMP menawarkan efisiensi deterministik dan stabil karena hanya membutuhkan satu lintasan terhadap teks dan tidak melakukan pencocokan ulang terhadap karakter yang telah diperiksa berkat penggunaan border function. Meskipun dalam lima pengujian tidak ada skenario yang memperlihatkan KMP secara eksplisit lebih unggul dibanding algoritma lainnya.

Selain pencarian eksak, sistem juga mendukung pencarian fuzzy yang ditangani dengan pendekatan Levenshtein Distance. Pada pengujian kedua, di mana keyword mengandung kesalahan ketik seperti "desgn", sistem tetap mampu memberikan hasil melalui fallback ke pencarian fuzzy. Meskipun tidak termasuk dalam tiga algoritma utama, keberadaan pencarian fuzzy ini menambah fleksibilitas sistem dalam menangani input yang tidak sempurna atau kesalahan ketik yang umum terjadi.

Secara keseluruhan, hasil pengujian menunjukkan bahwa pemilihan algoritma terbaik sangat bergantung pada kondisi pencarian. Boyer-Moore cocok untuk pencarian cepat terhadap satu keyword, Aho–Corasick cocok untuk pencarian terhadap banyak keyword, dan Knuth-Morris-Pratt cocok untuk pencarian tunggal yang konsisten pada dokumen yang tidak terlalu kompleks. Dengan adanya sistem perhitungan skor relevansi berdasarkan jumlah kecocokan dan frekuensi kemunculan keyword dalam dokumen, aplikasi ATS ini mampu memberikan hasil pencarian yang terurut dan informatif, sehingga sangat membantu dalam proses seleksi kandidat secara otomatis dan efisien.

BAB V

KESIMPULAN, SARAN, DAN REFLEKSI

5.1 Kesimpulan

Melalui Tugas Besar 3 IF2211 Strategi Algoritma ini, telah berhasil dikembangkan sebuah sistem Applicant Tracking System (ATS) berbasis aplikasi desktop yang mampu melakukan pencarian keyword dalam dokumen Curriculum Vitae (CV) digital menggunakan algoritma pencocokan string. Tiga algoritma utama yang diimplementasikan, yaitu Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), dan Aho-Corasick, masing-masing menunjukkan performa yang optimal dalam kondisi tertentu. Boyer-Moore paling efisien untuk pencarian tunggal, Aho-Corasick sangat unggul dalam pencarian banyak keyword sekaligus, sedangkan KMP memberikan hasil yang konsisten pada pencarian sederhana. Selain itu, sistem juga dilengkapi dengan fallback pencarian fuzzy menggunakan Levenshtein Distance yang mampu menangani kesalahan ketik pada input keyword maupun CV, sehingga menjadikan sistem lebih adaptif dan fleksibel.

Secara keseluruhan, aplikasi ini tidak hanya mampu menyaring informasi dari dokumen CV dengan cepat dan akurat, tetapi juga memberikan hasil pencarian yang terurut berdasarkan skor relevansi. Fitur interaktif seperti pemilihan algoritma, pengaturan jumlah hasil, dan tampilan detail CV menjadikan aplikasi ini sangat bermanfaat untuk proses rekrutmen digital yang efektif.

5.2 Saran

Dalam pelaksanaan tugas besar ini, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan program di masa depan:

1. Tidak menunda-nunda dalam mengerjakan Tugas Besar agar dapat lebih maksimal dalam mengeksplorasi seluruh spesifikasi wajib dan bonus.
2. Mengeksplorasi berbagai algoritma enkripsi dan melakukan perbandingan dari segi aspek keamanan, efisiensi, dan kompleksitas implementasi, sehingga dapat diterapkan algoritma yang paling sesuai untuk melindungi data pengguna dalam aplikasi.
3. Lebih teliti dan cermat dalam merancang pola *regex*, mengingat format dan struktur penulisan CV yang beragam dapat menyebabkan hasil ekstraksi informasi menjadi tidak konsisten jika pola tidak disesuaikan dengan variasi tersebut.

5.3 Refleksi

Dalam pengerjaan Tugas Besar 3 ini, kami belajar bahwa implementasi algoritma pencocokan string seperti Knuth-Morris-Pratt, Boyer-Moore, dan Aho-Corasick bukan hanya soal memahami konsep teoritisnya, tetapi juga berkaitan erat dengan bagaimana

algoritma tersebut diintegrasikan dalam sistem nyata yang memproses data tidak terstruktur, seperti CV dalam format PDF. Tantangan utama yang kami hadapi bukan hanya terletak pada implementasi logika algoritma, tetapi juga dalam merancang arsitektur sistem yang modular, membangun antarmuka pengguna yang intuitif, serta memastikan efisiensi dalam pemrosesan data. Salah satu hambatan teknis yang cukup signifikan adalah proses ekstraksi data menggunakan regular expression (regex), terutama dalam menyesuaikan pola pencarian terhadap berbagai variasi format penulisan di CV. Kesulitan ini mendorong kami untuk lebih teliti dalam mendesain pola regex dan mengoptimalkan proses ekstraksi agar tetap akurat.

Bekerja dalam kelompok juga mengajarkan kami pentingnya melakukan manajemen waktu yang baik serta komunikasi yang konsisten agar proses pengembangan dapat berjalan efektif. Selain mengimplementasikan fitur utama sistem ATS, kami juga mengeksplorasi fitur bonus seperti fallback pencarian fuzzy menggunakan Levenshtein Distance, serta penerapan enkripsi data untuk menjaga keamanan informasi profil pelamar. Eksplorasi enkripsi ini memperkenalkan kami pada praktik dasar pengamanan data dan menumbuhkan kesadaran akan pentingnya menjaga kerahasiaan informasi sensitif dalam sistem digital, terutama pada konteks rekrutmen. Kami menyadari bahwa pemilihan algoritma yang tepat dan proses pengujian yang menyeluruh sangat krusial untuk memastikan performa sistem tetap optimal dalam berbagai skenario pencarian. Secara keseluruhan, tugas besar ini tidak hanya memperkuat pemahaman kami terhadap algoritma pencocokan pola, tetapi juga memperluas wawasan teknis dan kolaboratif kami dalam membangun aplikasi ATS yang fungsional, aman, dan siap untuk dikembangkan lebih lanjut.

LAMPIRAN

Tautan Repository Github

Berikut tautan repository github kelompok kami untuk Tugas Besar 3 IF2211 Strategi Algoritma :

https://github.com/carllix/Tubes3_MtringSatching

Tautan Video

Berikut tautan video kelompok kami untuk Tugas Besar 3 IF2211 Strategi Algoritma :

Tabel Kelengkapan Spesifikasi

No	Poin	Ya	Tidak
1	Aplikasi dapat dijalankan.	✓	
2	Aplikasi menggunakan basis data berbasis SQL dan berjalan dengan lancar.	✓	
3	Aplikasi dapat mengekstrak informasi penting menggunakan Regular Expression (Regex).	✓	
4	Algoritma Knuth-Morris-Pratt (KMP) dan Boyer-Moore (BM) dapat menemukan kata kunci dengan benar.	✓	
5	Algoritma Levenshtein Distance dapat mengukur kemiripan kata kunci dengan benar.	✓	
6	Aplikasi dapat menampilkan summary CV applicant.	✓	
7	Aplikasi dapat menampilkan CV applicant secara keseluruhan.	✓	
8	Membuat laporan sesuai dengan spesifikasi.	✓	
9	Membuat bonus enkripsi data profil applicant.	✓	
10	Membuat bonus algoritma Aho-Corasick.	✓	

11	Membuat bonus video dan diunggah pada Youtube.	✓	
----	--	---	--

DAFTAR PUSTAKA

- Amazon Web Services. 2025. *What is an API?* <https://aws.amazon.com/what-is/api/> (Diakses pada 07 Mei 2025).
- Gin Contributors. 2025. *Gin*. <https://gin-gonic.com/> (Diakses pada 07 Mei 2025).
- MDN Web Docs. 2025. *Client-Server Architecture*. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview (Diakses pada 07 Mei 2025).
- Meta. 2025. *React Documentation*. <https://react.dev/> (Diakses pada 07 Mei 2025).
- Munir, Rinaldi. 2025. *Breadth/Depth First Search (BFS/DFS) (Bagian 1)*. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/13-BFS-DFS-\(2025\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/13-BFS-DFS-(2025)-Bagian1.pdf) (Diakses pada 07 Mei 2025).
- Munir, Rinaldi. 2025. *Breadth/Depth First Search (BFS/DFS) (Bagian 2)*. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/14-BFS-DFS-\(2025\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/14-BFS-DFS-(2025)-Bagian2.pdf) (Diakses pada 07 Mei 2025).
- PuerkitoBio. 2025. *goquery*. <https://github.com/PuerkitoBio/goquery> (Diakses pada 07 Mei 2025).
- The Go Authors. 2025. *Effective Go: Concurrency*. https://go.dev/doc/effective_go#concurrency (Diakses pada 07 Mei 2025).
- The Go Authors. 2025. *Golang Documentation*. <https://go.dev/doc/> (Diakses pada 07 Mei 2025).
- Vercel. 2025. *Next Documentation*. <https://nextjs.org/docs> (Diakses pada 07 Mei 2025).